Ant Colony Optimization for Job Shop Scheduling

Updated technical paper for the modern ACO-JSSP solver and browser visualizer

Adam DeJans Jr. | June 2026 | github.com/addejans/ACO-JSSP | addejans.github.io/ACO-JSSP/

Abstract. This paper documents a modernized Ant Colony Optimization implementation for the Job Shop Scheduling Problem. The repository started as a 2017 research prototype and now includes a Python 3 solver, deterministic tests, generated problem instances, a browser Gantt visualizer, and GitHub Pages deployment. The goal is not to prove global optimality. The goal is to provide a practical, inspectable demonstration of probabilistic construction, pheromone learning, exploitation, local search, and schedule decoding for a constrained sequencing problem.

Keywords: Ant Colony Optimization, job shop scheduling, metaheuristics, Gantt chart, operations research, local search, GitHub Pages.

1. Motivation and problem context

The Job Shop Scheduling Problem is a compact way to explain one of the hardest practical realities in operations: every job follows a route, every machine can process only one operation at a time, and every local sequencing choice can create downstream congestion. The objective in this repository is makespan minimization: finish the last operation as early as possible while respecting job-precedence and machine-capacity constraints.

Exact mathematical optimization formulations exist, but the search space grows quickly as jobs and machines increase. This makes JSSP a useful setting for metaheuristics. ACO converts scheduling into repeated constructive search: ants build sequences, good sequences receive more pheromone, and future ants are biased toward choices that historically produced shorter schedules.

Design goal: make the algorithm understandable, runnable, and visual before optimizing for industrial-scale performance.

2. Job shop scheduling model

A job shop instance is represented as a set of jobs. Each job is a sequence of operations. Each operation has a required machine and processing duration. A feasible schedule must satisfy job precedence and machine capacity.

Concept	Representation	Purpose
Job	Index j in jobs	Route processed in order
Operation	(machine, duration)	One processing step
Job order	Sequence of job IDs	Constructive representation built by ants
Schedule	Start/end per operation	Decoded feasible solution
Objective	Makespan	Completion time of the last operation

The modern solver uses a job-order encoding. Each time a job ID appears, the decoder schedules the next unscheduled operation for that job at the earliest feasible time. This guarantees job precedence by construction and lets the decoder enforce machine feasibility.

3. Ant Colony Optimization technique

ACO is a population-based constructive metaheuristic. Each ant incrementally builds a complete solution. The choice of the next job is random but biased by pheromone and a heuristic estimate of desirability.

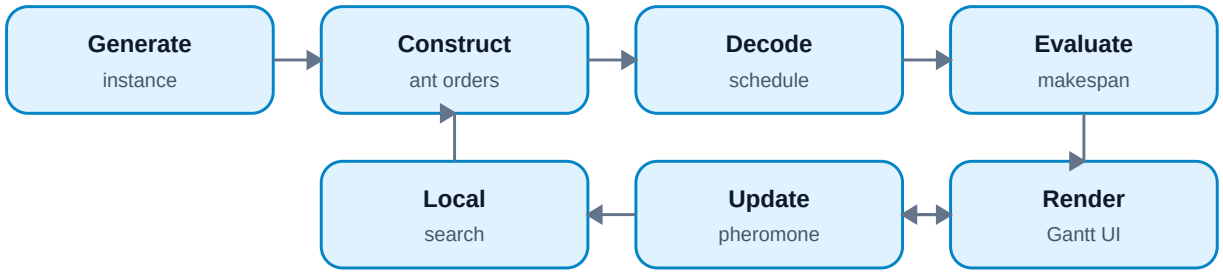

Figure 1. Updated ACO-JSSP flow from generated instance to schedule visualization.

- Generate or load a JSSP instance.
- Initialize pheromone values for transitions between job selections.
- Let each ant construct a full job-order sequence.
- Decode the job order into a feasible active schedule.
- Improve the constructed solution with pair-swap local search.
- Evaporate old pheromone and reinforce the iteration-best and global-best orders.
- Return the best schedule found and render it as a Gantt chart.

This is not a proof of optimality. It is a demonstration of improving stochastic search over a constrained scheduling space.

4. Modern implementation

The updated Python solver is dependency-free and uses standard-library data structures. The original repository scripts used Python 2 syntax and an old Plotly API. The modern implementation adds a clean object model, deterministic random seeds, JSON output, tests, and CLI controls.

File	Role
src/aco_jssp.py	Modern Python 3 ACO solver and CLI
tests/test_aco_jssp.py	Schedule validity and determinism tests
docs/index.html	Browser visualizer and Gantt chart UI
ERRORS_AND_FIXES.md	Review notes and corrected issues
.github/workflows/pages.yml	GitHub Pages deployment workflow
.github/workflows/ci.yml	Unit test workflow

```
    for each iteration:
        for each ant:
            order = construct_job_order(pheromone, heuristic)
            schedule = decode(order)
            improved = local_search(schedule)
            update iteration_best and global_best
        evaporate pheromone
        deposit pheromone on iteration_best
        deposit elite pheromone on global_best
```

5. Browser visualizer and user interface

The visualizer is a static HTML/JavaScript application deployed through GitHub Pages. It runs without a server and without external visualization libraries. The user controls problem size and solver parameters directly in the sidebar. The output is a machine-by-machine Gantt chart plus JSON for the resulting schedule.

Control	Meaning
Jobs	Number of jobs in the generated instance
Machines	Number of machines visited by each job
Ants	Number of constructed solutions per iteration
Iterations	Number of ACO learning rounds
Local search	Pair-swap improvement budget
Seed	Reproducibility control
Min/Max duration	Range of operation processing times

The UI is intentionally simple: choose numbers, run the optimizer, inspect the schedule. The original/generated split was removed because the instance-size controls already define the problem.

Live demo: https://addejans.github.io/ACO-JSSP/

6. Algorithmic improvements

The updated algorithm improves both software reliability and search quality. It is still compact, but it is no longer a direct mechanical port of the 2017 prototype.

Improvement	Why it matters
Generated instances	Users can run more jobs and machines instead of one fixed problem
Remaining-work heuristic	Construction considers downstream work, not just the next duration
Exploration/exploitation mix	Balances random search with strong weighted moves
Pair-swap local search	Improves constructed job orders before pheromone reinforcement
Elite reinforcement	Preserves signal from the global best solution
Pheromone floor	Avoids transitions disappearing permanently
Seeded randomness	Makes examples reproducible for testing and explanation

7. Testing and validation

The tests focus on correctness properties that should always hold regardless of stochastic search quality. A candidate schedule must satisfy job precedence, prevent machine overlap, return a complete set of operations, and behave deterministically when the seed is fixed.

- Machine non-overlap validation
- Job precedence validation
- Complete schedule length checks
- Deterministic output for fixed seeds
- Generated instance reproducibility
- Larger generated schedule smoke test

```
python -m unittest discover -s tests -v
```

These tests do not prove optimality. They prove that generated schedules are structurally valid and that the implementation is stable enough for demonstration and further development.

8. Limitations and extension roadmap

The project is a clear, educational implementation rather than a production-grade optimizer. It does not include benchmark calibration, optimality bounds, disjunctive MILP comparisons, parallel ant construction, advanced neighborhood search, or industrial data integrations. Those are natural next steps.

Extension	Expected benefit
Benchmark library support	Compare against standard JSSP instances
Critical path local search	Focus improvement on operations that drive makespan
Machine-level pheromones	Learn sequencing preferences specific to each machine
Dispatching-rule hybrids	Blend ACO with SPT, LPT, EDD, or slack-based rules
MILP baseline	Measure solution quality against exact or time-limited optimization
Parallel ants	Improve runtime on larger instances
Export CSV	Make UI output easier to reuse

The practical lesson is that metaheuristics become much more useful when paired with validation, visualization, and a clean problem representation.

Appendix. Historical code review notes

- The legacy scripts are retained for historical reference.
- The old implementation used Python 2 syntax and deprecated Plotly APIs.
- Hard-coded Plotly credentials were present in the historical file and should be considered compromised if still active.
- The new implementation avoids external plotting credentials and uses static browser rendering.
- New work should start from src/aco_jssp.py rather than the legacy files.